Caddie

Technical Guide

Tadhg Donnelly | 22480084 | tadhg.donnelly6@mail.dcu.ie

Gavin Szumlicki | 22511509 | gavin.szumlicki2@mail.dcu.ie

Boualem Benatallah | boualem.benatallah@dcu.ie

15/04/2026

Abstract

Modern teams and individuals work with an increasing amount of digital documentation, from onboarding guides and wikis to various web resources, yet having information available does not mean it is easy to find or act on when needed. Caddie was developed as a web-based onboarding and documentation support platform to address this problem. By bringing fragmented knowledge into collaborative workspaces called hubs, users can upload files, web links, and YouTube sources, allowing their contents to be searched semantically and used as context for retrieval-augmented chat, with answers tied back to the original material through citations. The same processed content is also used to generate more practical follow-up artefacts such as reminders, frequently asked questions, and step-by-step guides. In this way, Caddie is intended to make documentation easier to access especially for onboarding but also within broader knowledge-sharing contexts.

Table of Contents

Abstract **2**
Table of Contents **2**
1. Introduction and Motivation **4**
1.1. Project Overview and Problem Definition 4
1.2. Objectives, Purpose, and Scope 5
1.3. Intended Users and Use Context 5
1.4. Success Criteria 6
2. Research **6**
2.1. Background Research 6
2.2. Review of Existing Solutions 6
2.3. Technology Selection 7
2.4. Research-Informed Design Decisions 7
3. System Design **8**
3.1. Overall Architecture 8
3.2. Frontend Design 9
3.3. Backend Design 9
3.4. Database Design 10
3.5. Authentication and Security Design 11
3.6. Integrations and External Services 11

4. Implementation **12**
4.1. Project Structure 12
4.2. Frontend Implementation 13
4.3. API Implementation 14
4.4. Worker Implementation 15
4.5. Database Implementation 16
4.6. Testing Strategy and Implementation 16
4.7. CI/CD Approach 17
5. Sample Code Flow **18**
5.1. Frontend Example 18
5.2. Backend Example 19
5.3. Worker Example 20
5.4. Explanation 21
6. Problems and Resolutions **22**
6.1. Challenges Encountered 22
6.2. Resolutions 22
6.3. Trade-offs and Compromises 23
7. Results **24**
7.1. Features Delivered 24
7.2. Test Results and Validation 25
8. Future Work **26**
8.1. Current Limitations 26
8.2. Potential Improvements 27
8.3. Long-term Expansion Opportunities 27
9. Conclusions **28**
10. Appendices **29**
10.1. System Architecture Diagram 29
10.2. Asynchronous Worker Processing Pipeline for Source Ingestion 30
10.3. Chat Request and Retrieval Flow 31
10.4. Database Schema 32

1. Introduction and Motivation

1.1. Project Overview and Problem Definition

Caddie is a web-based platform that helps teams manage and use documentation more effectively. Rather than acting as a passive storage system, Caddie is intended to function as an active assistant that helps users organise information, retrieve it conversationally, and act on it through reminders and guides.

The core concept behind Caddie is the use of collaborative workspaces called hubs. Within a hub, users can upload files, submit web links, and manage a shared collection of sources related to a team, project, or onboarding process. These sources are processed through an ingestion pipeline so their contents can be searched semantically and used to ground chat responses in the original material.

Caddie was developed to address the difficulty of finding and using important information when documentation is fragmented across multiple sources. In many organisations, onboarding guides, technical manuals, policies, meeting notes, and team instructions are spread across cloud drives, wikis, PDFs, chat platforms, and internal websites. As a result, users often spend significant time locating information, checking whether it is current, and deciding which source is authoritative. This issue is especially pronounced during onboarding, where new users must absorb a large amount of unfamiliar material in a short period of time.

Existing solutions often address only part of the problem. Traditional knowledge bases provide storage and structure, but can be slow to search, difficult to maintain, and intimidating for casual users. General AI chat tools may provide faster answers, but they are not always grounded in a trusted set of sources, which creates concerns around accuracy. Caddie was designed to bridge this gap by combining structured source management with conversational, citation-based retrieval inside a collaborative workspace.

1.2. Objectives, Purpose, and Scope

The main objective of Caddie was to build a system that transforms static documentation into a more searchable, interactive, and actionable resource. Rather than simply storing documents, the platform was intended to help users organise their information into hubs, ingest multiple source types, retrieve answers through natural language questions, and generate useful follow-up actions such as Reminders and FAQs.

A second objective was to support collaboration. Documentation is often maintained by more than one person, particularly in workplaces where managers, HR staff, team leads, and project members all contribute knowledge. Caddie therefore needed to support shared hubs, role-based access, and workflows that allow teams to collectively maintain knowledge.

The purpose of this Technical Guide is to document the engineering design and implementation of Caddie. It explains why the system was created, the research that informed its development, the architectural decisions made during the project, and the technologies used to build the final solution. This document focuses on the technical aspects of the project, including system architecture, frontend and backend design, data handling, worker-based ingestion, authentication, testing, and deployment.

1.3. Intended Users and Use Context

Caddie is primarily intended for users operating in documentation-heavy environments. This includes new employees undergoing onboarding, managers or team leads responsible for organising documentation, HR staff maintaining internal resources, and project teams who need quick access to guides, procedures, or reference material. The common thread across these groups is that they depend on large amounts of written information but often lack an efficient way to retrieve it at the point of need.

Although the system is especially well suited to workplace onboarding, its use context is broader than professional environments alone. The same information-access problem appears in university projects, volunteer organisations, and any collaborative setting where documentation accumulates over time. Caddie is therefore designed as a general knowledge support platform with a professional focus.

1.4. Success Criteria

Caddie is successful if authenticated users can create and manage hubs, upload supported sources, query them through chat, and receive source-grounded answers. It must also support collaboration through shared access roles and provide practical outputs such as reminders, guides, and FAQs. From a technical perspective, success also requires usability, reliability, secure handling of authentication and storage, and an architecture that remains maintainable as the system grows.

2. Research

2.1. Background Research

Initial research examined how teams manage onboarding and internal knowledge. A consistent pattern was that documentation often exists in large quantities but remains difficult to use in practice. Users may know the information exists, yet still spend time searching through files, links, and message history to locate it. This slows onboarding, reduces efficiency, and increases reliance on incomplete or outdated knowledge. These findings also matched experiences observed during internships.

The research also showed that recent advances in retrieval-augmented generation have made it possible to treat documentation as an active knowledge source rather than a static archive.This capability became a central foundation for Caddie, as it offered a practical way to bridge the gap between document repositories and intelligent assistance.

2.2. Review of Existing Solutions

Existing tools generally fall into several incomplete categories. Traditional knowledge management platforms such as Confluence and SharePoint provide structured storage and collaboration, but still depend heavily on keyword search and manual navigation. Lightweight note-taking tools such as Evernote or Obsidian are flexible, but are less suited to shared, role-based workflows in professional settings.

AI chat tools such as ChatGPT and Claude offer a more natural interaction model, but generic versions are not inherently connected to a trusted set of organisational

documents. Enterprise products such as Microsoft Copilot attempt to combine AI with workplace knowledge, but can be costly and may be less practical for smaller teams or projects.

2.3. Technology Selection

Caddie's technology stack was selected based on project requirements, integration suitability, and team familiarity. The frontend uses Next.js, React, and TypeScript to support an interactive, component-based interface for hubs, uploads, chat, and dashboards. TypeScript also improves maintainability through stronger typing.

The backend uses FastAPI for strongly typed API development and Supabase for Postgres storage, authentication, file storage, and row-level security. Redis and Celery support asynchronous workflows such as ingestion and reminder processing. OpenAI powers the retrieval-assisted chat layer, while vector embeddings enable semantic search over processed content. Together, these technologies supported a modular architecture in which the frontend, API, and worker could evolve separately while still functioning as one integrated system.

2.4. Research-Informed Design Decisions

The research phase shaped several core design choices. Because trust and traceability were identified as key concerns in AI-assisted knowledge systems, Caddie uses source-grounded retrieval so answers can be linked back to user-provided materials. Because onboarding and documentation are collaborative by nature, the system is organised around shared hubs rather than personal document spaces.

The technical architecture was also influenced by these findings. Since uploaded files, web pages, embeddings, and reminder extraction could not be processed reliably within standard API requests, Caddie uses a worker-based pipeline with Celery and Redis. FastAPI was chosen over heavier backend frameworks such as Django because it provided a more lightweight way to build strongly typed service endpoints without the extra overhead of a full server-rendered framework.

3. System Design

3.1. Overall Architecture

Caddie's architecture is composed of a web frontend, an API service, a background worker layer, and managed backend services. The web application provides the user interface for authentication, hub management, uploads, chat, and every other user-facing feature. The API acts as the main orchestration layer between the frontend and the system's data and AI capabilities. Background tasks are delegated to Celery workers so that long-running ingestion and reminder-processing operations do not block the user experience. Supabase provides persistent storage, authentication, and access control, while Redis is used to coordinate asynchronous task execution.

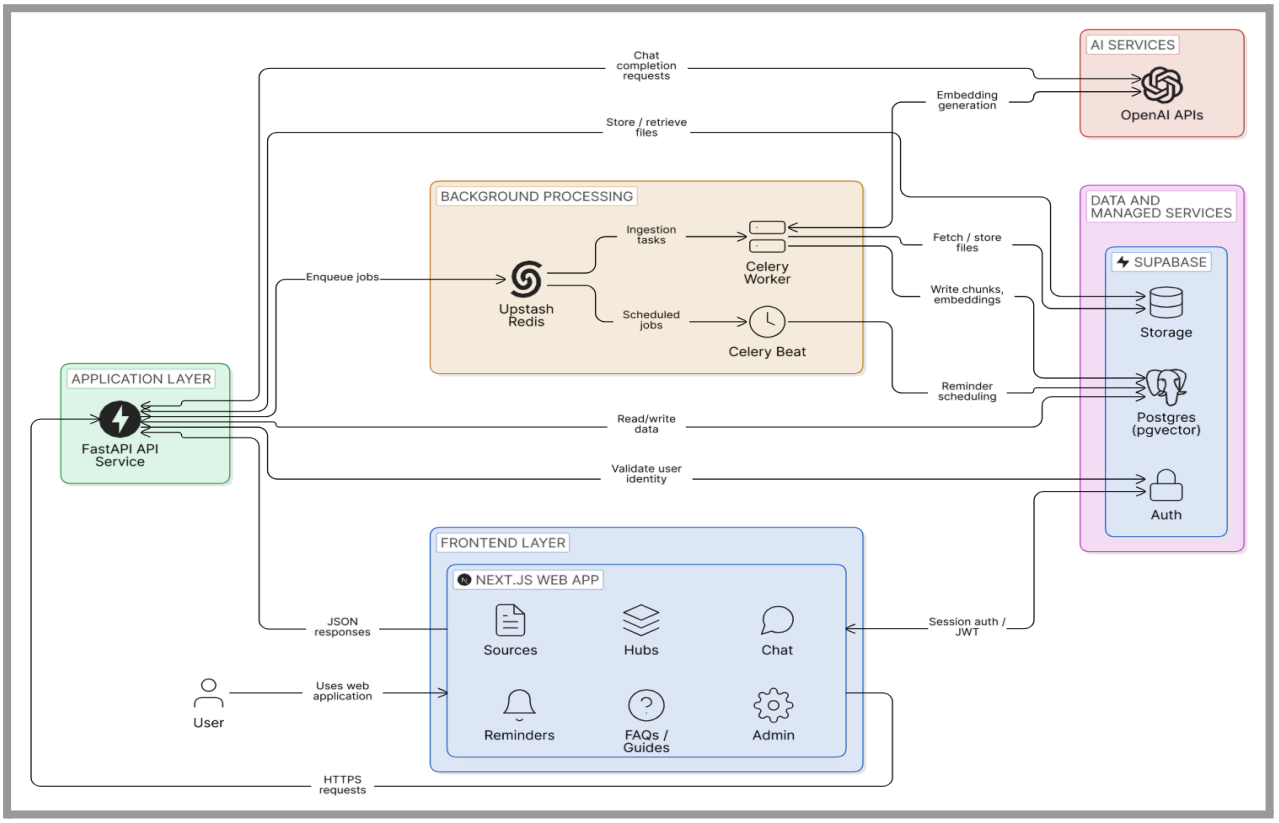


Figure 3.1 : Caddie High-Level System Architecture Diagram

This separation was chosen to improve modularity and maintainability. Each layer has a clear responsibility. The architecture also supports realistic production deployment because the frontend, API, and worker services can be hosted independently and scaled according to their own needs. This is particularly important in a system like Caddie, where chat and ingestion workloads differ significantly from static page rendering.

3.2. Frontend Design

The frontend was designed as a single-page style web application using Next.js and React. Its role is not only to render pages, but also to manage the interactive workflow involving authentication state, hub navigation, uploads, chat sessions, reminders, and role-sensitive actions. The application is organised around reusable components and route-based views, with individual pages and panels handling different aspects of hub interaction such as sources, members, admin controls, and chat. This structure keeps the UI modular, maintainable, and easily extensible.

Particular attention was given to keeping the user experience responsive during operations that may take time, such as source uploads or ingestion completion. The interface makes use of statuses, loading states and skeletons, and clear errors so that users are not left without feedback. Frontend logic also reflects role-based permissions, ensuring that controls for editing, uploading, archiving, or moderation are shown only when appropriate.

3.3. Backend Design

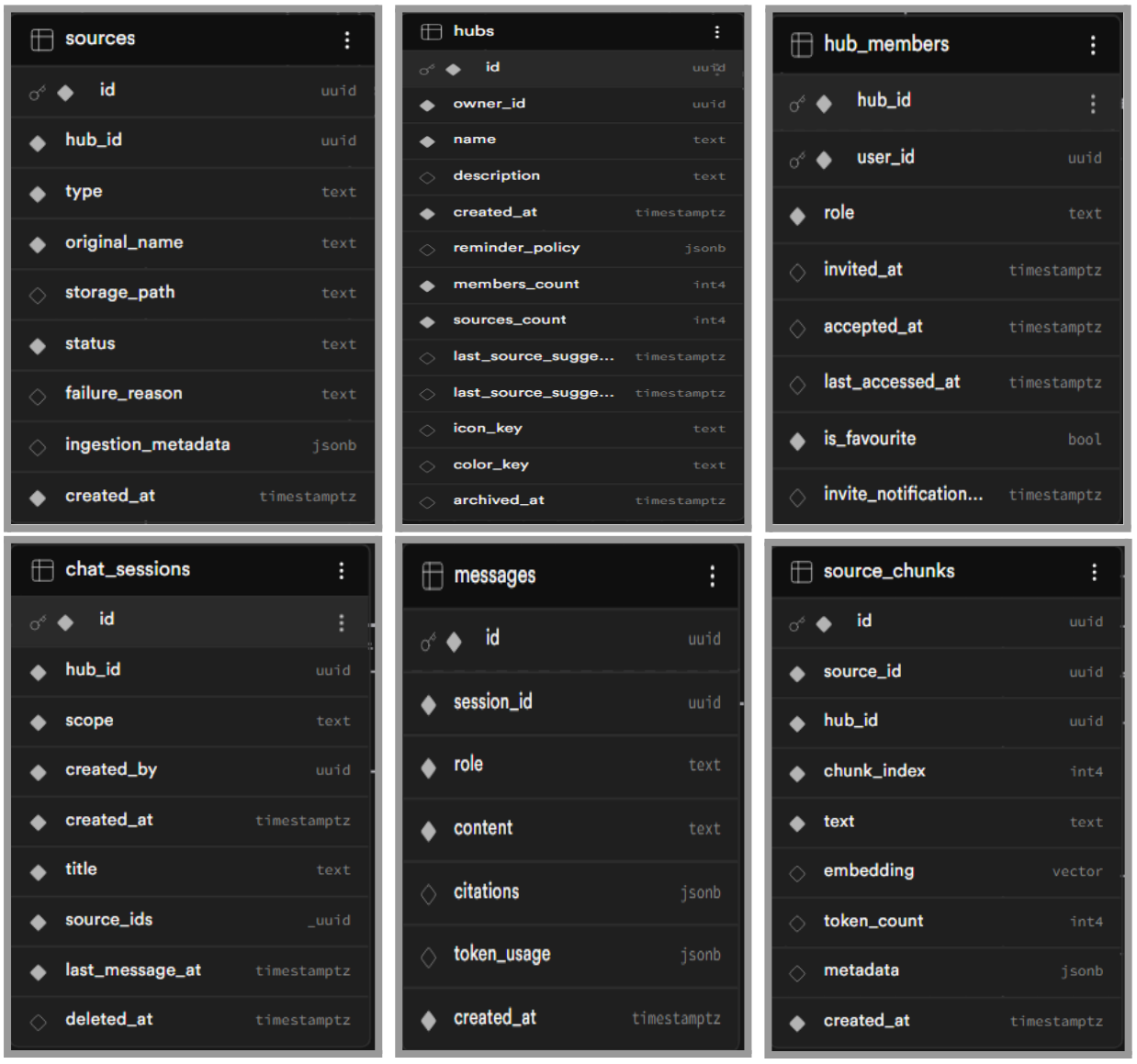
The backend was designed as a FastAPI service exposing endpoints for hubs, sources, chat, reminders, and administrative functionality. Rather than embedding large amounts of business logic directly inside route handlers, the system uses service-level abstractions to keep responsibilities clearer and make the codebase easier to test. This service-oriented backend design helps separate request validation, database operations, retrieval logic, and orchestration of external integrations such as storage and AI calls.

The backend also acts as the coordination point between user requests and asynchronous worker processing. For example, when a source is created from a file or URL, the API records the source metadata, triggers the appropriate ingestion task, and returns control to the frontend without waiting for the entire extraction and embedding process to finish. This design allows the backend to remain responsive and focused on API-level concerns while delegating heavier processing to the worker layer. It also increases the project's extensibility, as new ingestion types or retrieval features can be introduced without redesigning the entire API.

3.4. Database Design

Caddie's data model was designed around the relationships needed to support collaborative knowledge management. Core entities include users, hubs, hub memberships, sources, chat sessions, messages, reminders, and related metadata. This structure allows users to belong to shared workspaces with defined roles, store multiple content sources per hub, and maintain chat histories and reminder records connected to a given context. Since the system depends on both ownership and collaboration, the model needed to support not just content storage but also access control and activity tracking.

Supabase Postgres was selected as the database foundation because it provides a robust relational model while also supporting features needed for the project, such as row-level security and vector-based retrieval support through PostgreSQL extensions. Storing embeddings and processed document metadata alongside relational records makes it possible to integrate semantic retrieval into the wider application rather than treating it as a disconnected subsystem. Below are some of the key tables as designed:



3.5. Authentication and Security Design

Caddie uses Supabase Auth to handle user authentication, which provides a managed and secure foundation for login and identity management. The frontend relies on authenticated sessions, while the backend validates user context to ensure that only authorised users can access or modify protected resources. This prevents unauthenticated access to hubs and ensures that shared workspaces remain restricted to valid collaborators.

Authorisation is enforced through a combination of application-level logic and database access constraints. Hubs use membership roles such as owner, admin, editor, and viewer to determine what actions each user may perform. These roles influence both what is exposed in the frontend interface and what operations the backend will accept. Additional security considerations include careful handling of storage access, separation between public and privileged keys, and the use of row-level security patterns to protect user-scoped data.

We also make use of several defensive controls beyond authentication and role-based authorisation. Input validation is used throughout the API to ensure that incoming data conforms to expected formats before it is processed, while filename sanitisation and controlled URL handling reduce the risk of unsafe or malformed source inputs. API rate limiting is used to protect the platform from abuse and excessive request volume, with separate limits for chat, source operations, reads, writes, and health checks. Additional protections include explicit CORS allowlists for non-local environments, separation between user-scoped tokens and the privileged service role key, and worker-side checks such as public-host validation and robots.txt enforcement for web ingestion.

3.6. Integrations and External Services

The project depends on several external services to provide its full functionality. Supabase is the most central integration, acting as the system of record for authentication, relational data, and object storage. OpenAI is used to support AI-assisted conversation and retrieval-based answer generation. Redis enables the background task queue used by Celery. Depending on source type, the worker may also interact with external web resources when ingesting URLs or extracting content from supported locations (i.e YouTube API). These integrations were chosen to avoid reinventing critical infrastructure while still allowing the project to focus on its core value without incurring scope-creep.

4. Implementation

4.1. Project Structure

The implementation of Caddie is organised as a multi-application repository with a clear separation between the frontend, backend, and worker systems. The codebase is divided into src/apps/web, src/apps/api, and src/apps/worker, with each application maintaining its own responsibilities, dependencies, and runtime configuration. In addition, shared contracts are used where necessary to keep communication between applications consistent. This structure allowed development to proceed in parallel across the main parts of the system while keeping the repository organised and easier to navigate. From an implementation perspective, this layout also made the project easier to maintain and test. Each application could be run independently during development, and issues could be isolated more quickly because frontend, API, and worker concerns were not mixed into a single codebase. This proved especially useful as the project grew in scope, since it allowed new features to be added without introducing unnecessary coupling between unrelated parts of the system.

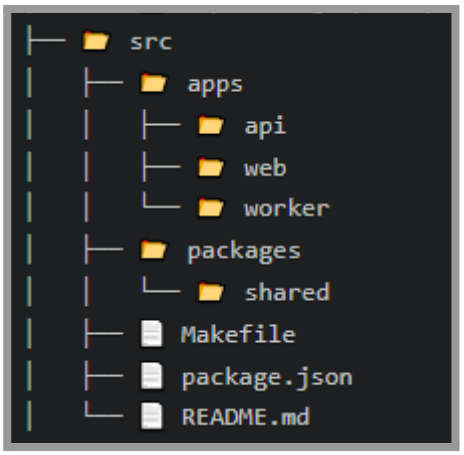


Figure 4.1 : File tree representation of source code repository, highlighting clear separation of main services

4.2. Frontend Implementation

The frontend was implemented using Next.js, React, and TypeScript, with the application structured around reusable components and route-based pages. Key interface areas include hub browsing, hub detail views, upload panels, chat sessions, reminder displays, dashboard views, FAQ and guide generation, authentication pages, and administrative controls. Reusable UI components were used extensively so that common patterns such as cards, modals, status messaging, and action panels could be shared across multiple parts of the application. A significant part of the frontend implementation focused on handling real user interaction states rather than static rendering alone. For example, uploads required progress feedback, pending states, and refresh behaviour once ingestion had completed. Similarly, dashboard and chat views needed to respond clearly to loading, success, and failure cases while remaining intuitive for the user.

4.3. API Implementation

The API organises routes around the main functional areas of the platform such as hubs, sources, chat, reminders, FAQs, guides, memberships, and moderation. Request validation and typed response models were used throughout to make endpoint behaviour more consistent and easier to debug. In practical terms, the API implementation acts as the point where user actions are translated into application behaviour. A request such as creating a source, generating a guide, or sending a chat message often requires more than a simple database write, and may involve validation, orchestration, context retrieval, and task queueing.

Validation and hardening were implemented directly in the API layer. FastAPI request schemas are used to validate fields such as names, filenames, titles, content, and submitted URLs before requests reach deeper business logic. Source filenames are sanitised before being stored, and invalid or malformed requests are rejected early with typed error responses. The API also applies Redis-backed rate limiting at route level, with different configurable thresholds for chat, source, read, write, and health endpoints.

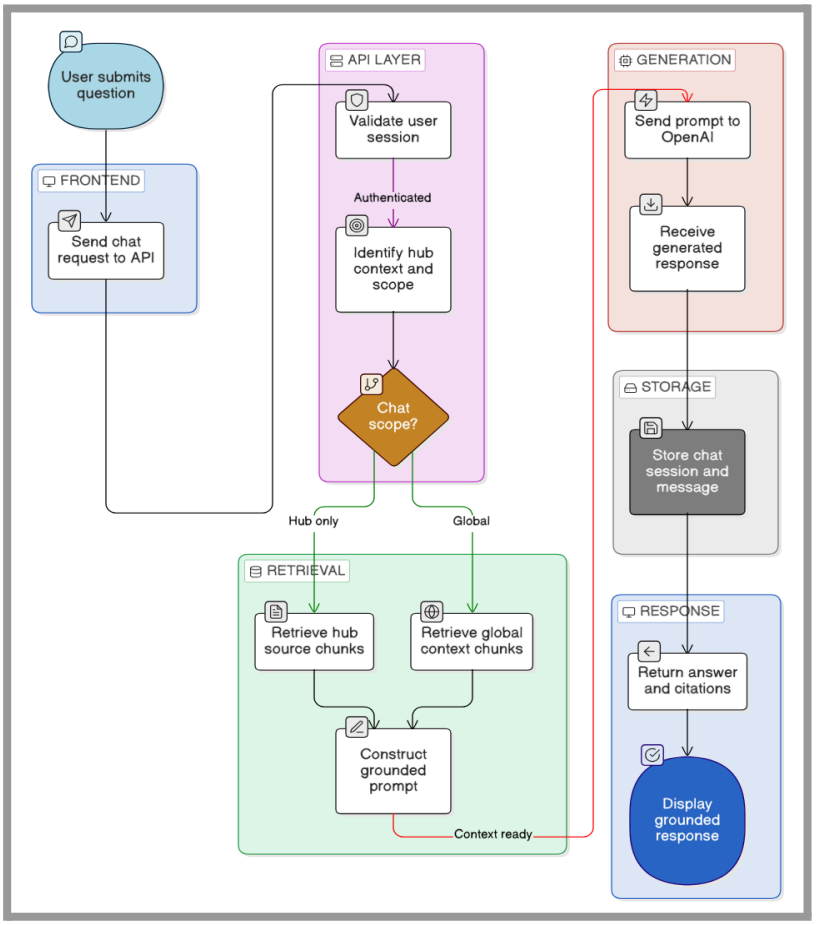


Figure 4.3 : Chat Request and Retrieval Flow

4.4. Worker Implementation

The worker implementation handles the background processing required for ingestion and scheduled reminder behaviour. Celery tasks were used for operations such as file ingestion, web ingestion, YouTube transcript handling, chunk generation, embedding generation, and source reprocessing. The worker implementation also includes validation and sanitisation steps specific to ingestion. For web sources, submitted URLs are checked to ensure they point to public hosts rather than private or local addresses. Retrieved HTML and transcript content are cleaned and normalised before chunking so that downstream storage and retrieval operate on a more consistent text representation.

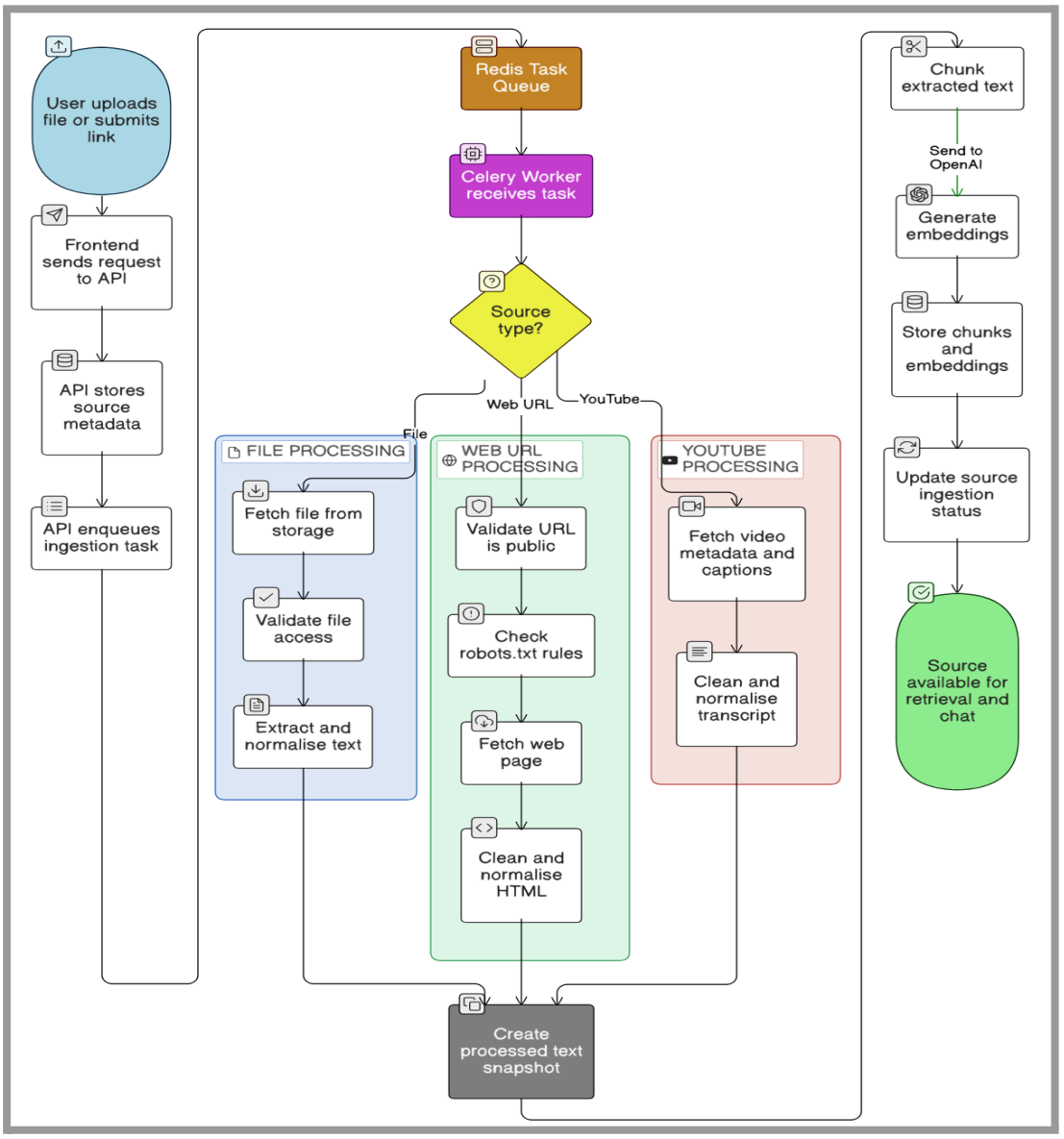


Figure 4.4 : Asynchronous Worker Processing Pipeline for Source Ingestion

4.5. Database Implementation

The database implementation was carried out using Supabase Postgres, with the schema extended to support both collaborative application data and retrieval-related content processing. Schema changes and additions were applied using SQL migrations. Core records such as hubs, memberships, sources, chats, reminders, FAQs, and guides are stored in relational form, while processed source outputs such as chunks and embeddings are stored in a way that supports semantic retrieval. This allowed the retrieval pipeline to remain part of the main application model rather than existing as an isolated subsystem.

A major implementation concern was preserving consistency between collaboration features and AI-supported features. For example, source ownership, hub membership, reminder visibility, and chat access all needed to respect the same permission boundaries. At the same time, processed source data had to remain linked back to its original hub and source records so that responses could be grounded and traceable. The resulting schema therefore had to support both conventional application relationships and the additional data required for vector-based retrieval.

4.6. Testing Strategy and Implementation

Testing was implemented at several levels across the project. On the frontend, unit and component tests were used to verify interface behaviour such as rendering, permissions, uploads, navigation, and error handling. On the API side, tests were used to validate route behaviour, service logic, and data-handling rules. Worker-side tests focused on ingestion and processing behaviour, helping ensure that source handling remained stable across different input types and task paths.

In addition to these lower-level tests, integration and end-to-end testing were used to validate real workflows across multiple parts of the system. This was particularly important for Caddie because the most valuable features, such as uploading a source and later chatting over it, depend on the frontend, backend, authentication, data storage, and worker pipeline all functioning together correctly. We go more into depth on testing in our Testing Report Document.

4.7. CI/CD Approach

Caddie's CI/CD setup was implemented through GitLab CI, with separate stages covering linting, testing, and build validation across the different applications. The pipeline includes frontend linting and type checking, web tests, backend and worker tests, build verification, and higher-level end-to-end validation. This provided an automated way to catch regressions before changes reached the main branch and helped maintain consistency across a codebase that spans several runtimes and services.

Deployment was implemented as a controlled promotion flow, with promote_production acting as a manual gate for deployment. GitLab acts as the primary source repository and CI control point, while GitHub is used as a deployment mirror for the hosted services. The frontend is deployed through Netlify, while the API, worker, and scheduler processes are deployed through Railway. Once successfully deployed via the promote_production job, a post_deploy_health job runs to verify the live services.

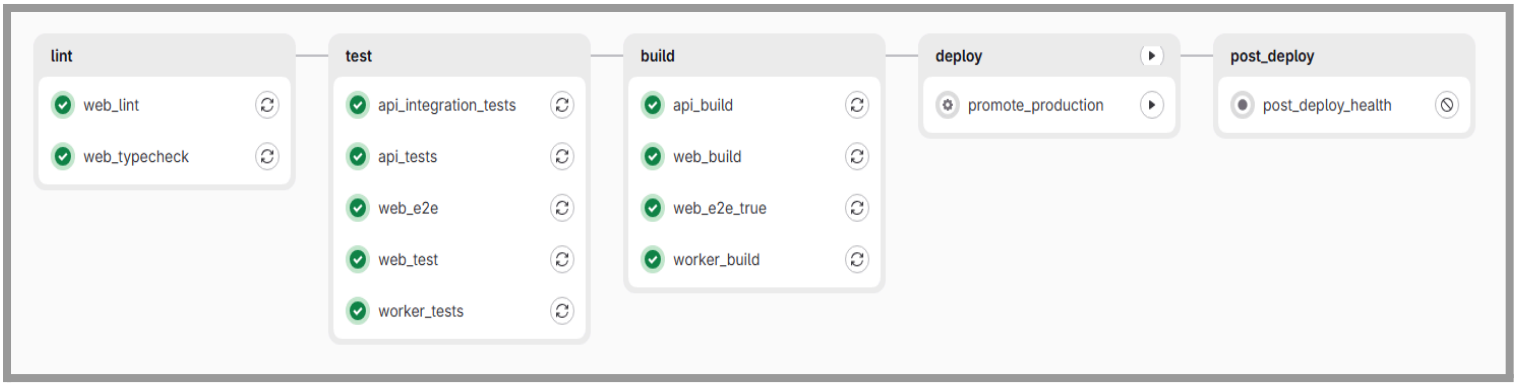


Figure 4.7: Gitlab CI/CD pipeline for Caddie, showing the staged validation flow and the controlled promotion to production

5. Sample Code Flow

Section 5 illustrates the flow of a web source through Caddie, from the moment a user submits a URL in the frontend to the point where that source becomes available for retrieval.

5.1. Frontend Example

The main frontend entry point for adding a source is AddSourceModal. It accepts file, web, and YouTube items and processes them one at a time using a simple per-item state model: pending, uploading, enqueuing, complete, and error. These states are reflected in the UI so users can track progress and see failures without blocking the rest of the batch.

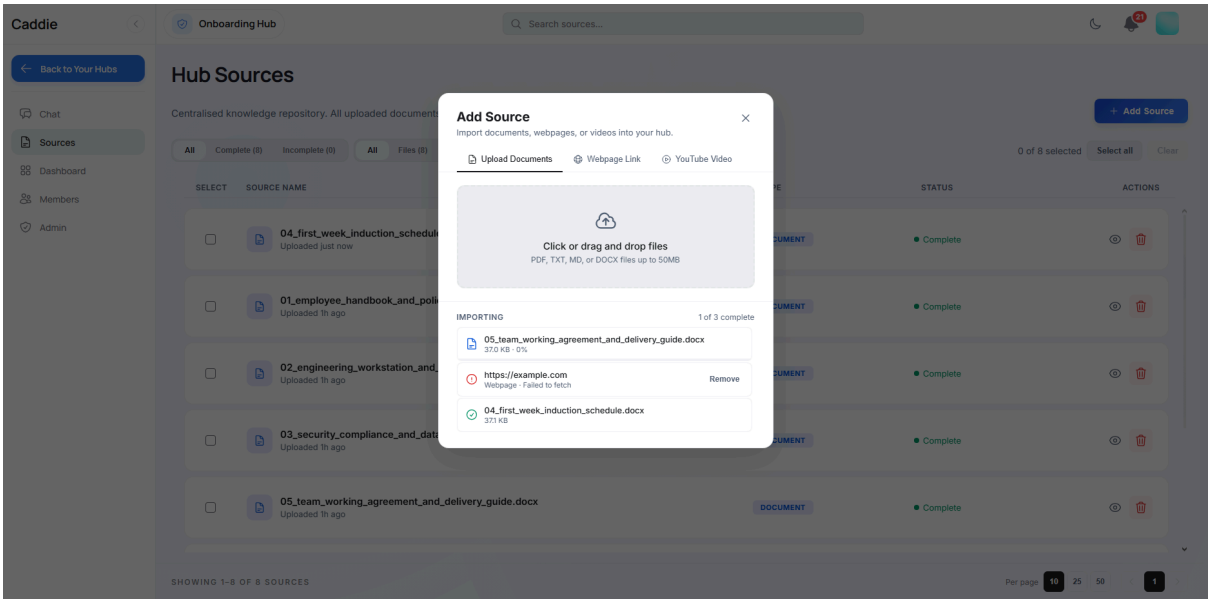


Figure 5.1: AddSourceModal

For web sources, the frontend only normalises the URL, calls the API, updates item state, and refreshes cached hub data:

```ts
let finalUrl = nextItem.url;
if (!/^https?:\/\//i.test(finalUrl)) {
  finalUrl = `https://${finalUrl}`;
}
setItemState(nextItem.id, "enqueuing");
await createWebSource({ hub_id: hubId, url: finalUrl });
```

```
setItemState(nextItem.id, "complete");

onRefresh();
```

If the user omits http:// or https://, the URL is corrected before submission. Each call to 'setItemState' advances the per-item state machine so the modal UI reflects the current stage of the batch. The 'createWebSource' helper posts to the API and returns as soon as the source row has been created. 'onRefresh()' invalidates cached hub data so the new source appears without reloading the page. The browser does not fetch the web page or wait for ingestion to finish; that work is handled asynchronously after the request is sent.

5.2. Backend Example

On the API side, web-source creation is handled by the sources router. FastAPI injects a request-scoped Supabase client and the authenticated user before the handler runs, so normal data access automatically respects row-level security.

```py
@router.post("/web", response_model=Source)

def create_web_source(payload: WebSourceCreate, client: Client,
current_user: CurrentUser) -> Source:

    source = store.create_web_source(client, payload)

    celery_app.send_task(

        "ingest_web_source",

        args=[source.id, source.hub_id, payload.url, source.storage_path],

    )

    store.log_activity(

        client, source.hub_id, current_user.id, "created", "source",
source.id, {"type": "web"},

    )

    return source
```

The handler creates the sources row, queues a Celery ingestion task, logs the activity, and returns immediately. The API does not fetch the page itself, which keeps request latency short and prevents remote sites from slowing the server.

5.3. Worker Example

The 'ingest_web_source' task is implemented in the worker and runs a small ingestion pipeline:

```
```py
safe_url = _web._validate_public_url(url)
raw, content_type, final_url = _web._fetch_url_content(safe_url)
text, title = _web._extract_web_text(raw, content_type)
text = _common._normalize_text(text)
pseudo_doc = _web._build_pseudo_doc(title, final_url or safe_url, crawl_at,
content_type, text)
_storage._upload_pseudo_doc(client, storage_path, pseudo_doc)
chunk_count = _ingest_text_for_source(
    client, source_id, hub_id, text, extra_metadata=extra_metadata,
)
```
```

Each step has a clear job. '_validate_public_url' rejects private, loopback, and link-local addresses so that a malicious or accidental internal URL cannot be used to coerce the worker into fetching an internal service. '_fetch_url_content' performs the HTTP request with timeouts and size limits, and can optionally enforce 'robots.txt' policy. '_extract_web_text' extracts readable body text from HTML or other content types. '_normalize_text' collapses whitespace and strips control characters so that downstream chunking and embedding behaves consistently across different source types.

'_build_pseudo_doc' and '_upload_pseudo_doc' then produce a normalised snapshot of the fetched page and write it back into Supabase Storage. This snapshot is important because it gives the system a stable copy of the extracted content that can be reprocessed later, without treating the original external URL as the sole source of truth. The final call, '_ingest_text_for_source', is where the content becomes retrievable. The

normalised text is split, embedded, and inserted using defaults that live in the worker configuration module:

- chunk size: 800 words with 150 words of overlap;
- embedding batch size: 64, using the configured embedding model;
- insert batch size: 100 rows into 'source_chunks'.

Once the inserts finish, the source row is updated with the chunk count and marked as 'complete'. Reminder candidate extraction is attempted on the same normalised text in the same pass, so any detectable due dates or actions are surfaced to the user for review without a second ingestion run.

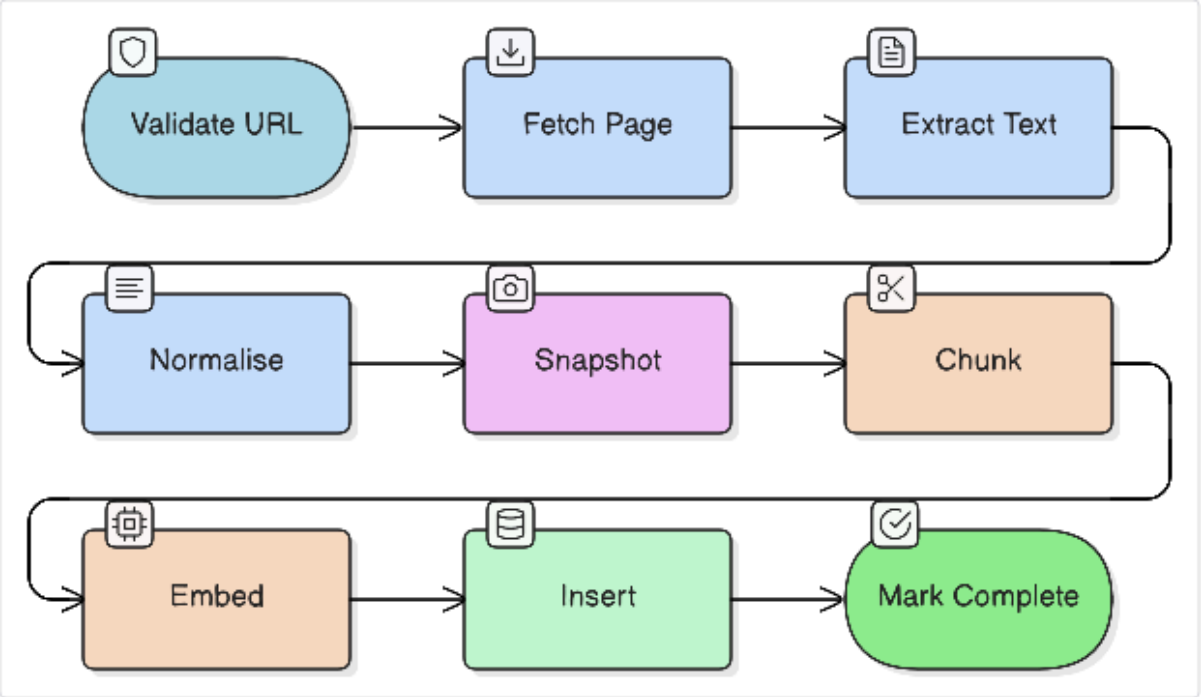


Figure 5.3: Pipeline strip diagram of the ingestion stages in Caddie

5.4. Explanation

The full web-source flow is:

1. The user submits a URL in AddSourceModal.
2. The frontend normalises the URL and sends it to the API.
3. The API creates a queued source record, enqueues a worker task, logs activity, and returns.
4. The worker validates and fetches the page, extracts and normalises the text, and stores a snapshot.

5. The worker chunks and embeds the text, stores it in source_chunks, and marks the source as complete.
6. The source can now be used in chat, analytics, FAQs, and guides.

6. Problems and Resolutions

6.1. Challenges Encountered

A key challenge was grounding. A documentation assistant is only useful if its answers can be traced back to real source material. Without this, plausible but incorrect answers would reduce trust in the system.

Another challenge was ingestion variability. Caddie supports files, web pages, and YouTube transcripts, but each source type can fail differently. Some files contain no extractable text, some websites block crawling, and some videos have no usable captions.

Permission consistency was also difficult. Access rules had to remain correct across the UI, API, and database as collaboration features expanded.

A final challenge was balancing answer quality, latency, and cost. Stronger models and more context improve answers, but increase both response time and operating cost.

6.2. Resolutions

Grounding was addressed by designing chat around retrieval and citations from the start. The chat router retrieves from 'source_chunks' with a configurable 'top_k' and 'min_similarity', diversifies the result set, and returns explicit citation objects that the frontend uses to render links back to the source. Users can scope chat to a hub, select individual 'sources' to retrieve over, and submit feedback on both the answer and individual citations. This means that even when the model produces a weak answer, the reader can inspect the material the answer is based on rather than having to trust the model blindly.

Ingestion variability was addressed by moving all fetching and extraction into the worker layer, defining explicit source statuses, and recording failure reasons on the source row itself. A user-submitted web URL that returns a timeout, a forbidden response, or an unparseable document becomes a failed source with a visible reason,

rather than a frozen request or a silently empty source. Because the statuses are persisted, the UI can offer targeted retry and refresh actions without reconstructing the state.

Permission consistency was addressed by enforcing access rules in three places at once rather than relying on any single layer. The frontend hides controls that the current role cannot use. The API routers use role-aware dependencies that reject operations before they reach the store. The Supabase clients created from the caller's token honour row-level security, which means that a request which bypasses a frontend check or slips past a router check still cannot see rows it should not see. Defence in depth is the only approach that remains correct under refactoring.

The quality, cost, and latency trade-off was addressed by tuning every stage of the chat pipeline rather than reaching for a bigger model. 'gpt-4o-mini' is the default for chat completion, which keeps per-query cost low without a meaningful drop in answer quality for documentation-style questions. Retrieval is capped by a configurable 'top_k' and a minimum similarity threshold, so the context window grows only as far as it needs to. Embeddings are computed once at ingestion and reused across every query rather than being regenerated per request, so the only model call on the hot path is the final completion itself.

6.3. Trade-offs and Compromises

Moving ingestion to the worker keeps the API responsive, but sources are not immediately available after upload. Users must wait until processing finishes. This was accepted because visible progress is preferable to long blocking requests.

Web and YouTube sources are also stored as pseudo-documents in Supabase Storage. This increases storage usage, but improves reprocessing, traceability, and citation stability.

Permission rules are enforced in multiple layers rather than one, which increases maintenance work but reduces the risk of access control bugs during refactoring.

7. Results

7.1. Features Delivered

Grouped by area, the shipped features are as follows:

- Collaboration: collaborative hubs with creation, editing, favouriting, archiving, and access tracking; role-based memberships with invitations, role changes, removal, and ownership transfer.
- Ingestion: source creation for uploaded files, generic web pages, and YouTube transcripts; per-source statuses with refresh, retry, and deletion; pseudo-document snapshots for web and YouTube sources; chunk inspection for debugging or review.
- Retrieval: chat with saved sessions, session search, per-source filtering, hub-only and broader scopes, inline citations, citation feedback, and answer feedback; configurable retrieval parameters exposed through environment variables.
- Knowledge artefacts: reminder candidate extraction from processed source text and full reminder CRUD with status transitions; generated FAQs with pin, edit, archive, and citation support; generated guides with per-step progress tracking and reordering.
- Administration: per-hub analytics summaries and trends; moderation flows for flagged AI content with review and action endpoints; AI-generated source suggestions that admins and editors can review before adoption; a hub activity feed populated by logging.

Each of these features is reachable from the web application and backed by dedicated API routes and, where relevant, by store-level tests.

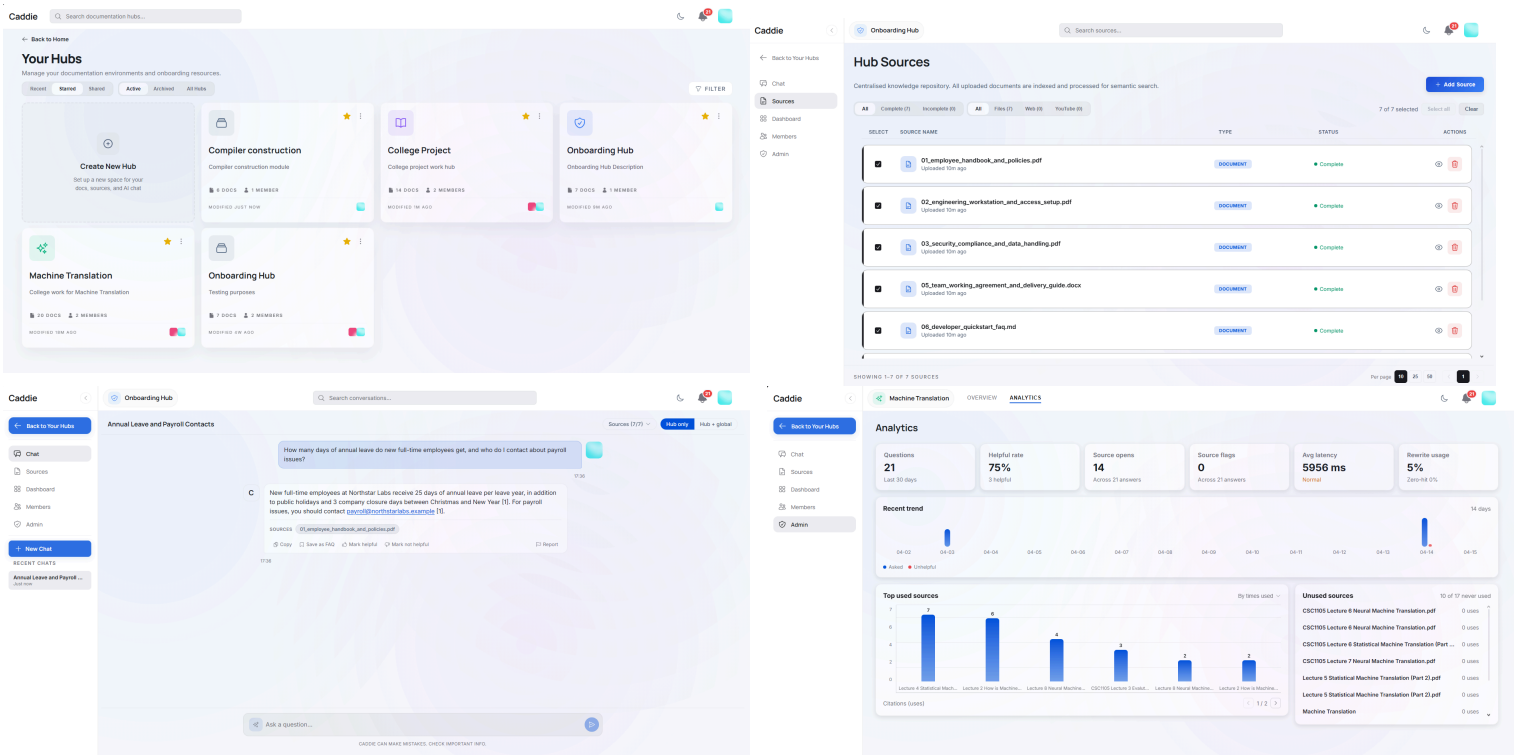


Figure 7.1: Representative screens from the Caddie web application: hubs list, hub detail, chat with citations, reminders calendar, guides with step progress, and analytics summary.

7.2. Test Results and Validation

Validation comes from both automated tests and broader pipeline checks. The strategy was deliberately layered: unit and component tests on the web side cover rendering, permissions, and form behaviour; pytest suites on the API and worker cover request validation, store logic, and ingestion paths; and end-to-end browser tests cover the cross-service flows that none of the lower layers can verify on their own. The local unit and component test runs collected for this manual revision, executed against the current main branch after pulling the latest changes, were:

- Web test suite (Vitest, 28 files): 130 tests passed.
- API test suite (pytest): 206 tests passed.
- Worker test suite (pytest): 22 tests passed.

```
Test Files  28 passed (28)
     Tests  130 passed (130)
  Start at  16:55:24
  Duration  9.06s (transform 3.79s, setup 21.09s, collect 20.48s, tests 26.94s, environment 35.50s, prepare 4.84s)
```

Figure 7.2a: Vitest results for the Caddie web test suite, showing 130 tests passing across 28 files.

```
========================================================== 22 passed, 12 warnings in 1.85s ==========================================================
========================================================== 206 passed, 15 warnings in 0.83s ==========================================================
```

Figure 7.2b: Pytest results for the Caddie API and worker test suites, showing 206 and 22 tests passing respectively.

These are supplemented by Playwright browser tests and the web_e2e_true GitLab CI job, which runs the web app, API, worker, and Redis together and validates common realistic use cases such as sign-in, upload, ingestion, and chat.

Above is an overview of the testing for this project to avoid repetition between documents. **We go into more detail about testing in our testing documentation.**

8. Future Work

8.1. Current Limitations

Caddie is fully functional, but some limitations remain in the current implementation. Chat responses are returned only once generation is complete, so longer answers can feel slower than necessary. Chat sessions are also user-owned rather than shared hub artefacts, which keeps the model simple but limits collaboration around previous conversations. Similarly, reminder delivery is currently limited to in-app notifications, so users must return to the platform to see them.

There are also some small practical limits in the ingestion pipeline. Web sources must be public and may be blocked by robots.txt, timeout limits, or page structures that do not extract cleanly. YouTube ingestion depends on captions being available and usable.

In addition, some assistive features such as source suggestions, FAQ generation, and guide generation are deliberately conservative. For example, suggested sources only run when a hub already has enough completed material, and only a small batch of suggestions are stored at a time. This improves reliability but keeps these features' scope narrower than a more ambitious production system.

8.2. Potential Improvements

The most immediate improvement would be streaming chat responses. The current chat pipeline already produces citations and grounded answers reliably, so the next step is improving the experience of receiving them. Streaming would make the interface feel faster without changing the underlying retrieval design.

A useful next improvement would be stronger hub hygiene features. Caddie could detect duplicate sources, suggest better source titles, flag stale material, or prompt users to merge overlapping content. This would improve retrieval quality without changing the core chat pipeline.

The system could also benefit from a reusable shared knowledge layer above individual hubs. Shared source libraries, hub templates, or organisation-level starter packs would reduce repeated setup and make onboarding faster for similar teams or projects.

Finally, reminder delivery could also be extended beyond the platform itself.The current notification structure already separates reminders from delivery channels, so adding email or similar delivery methods would build on the existing model rather than replace it. This would make the reminder system more useful for actual onboarding or deadline tracking.

8.3. Long-term Expansion Opportunities

Longer term, Caddie could grow in directions that would make it closer to an enterprise knowledge platform.

Integrations with existing systems such as Confluence, SharePoint, Google Drive, Notion, or Slack would let Caddie ingest documentation without requiring users to re-upload it. Document version tracking and change alerts would let users see when the source they relied on has been superseded. Organisation-wide administration and single sign-on would make the product viable for corporate deployment rather than only team

use. Personalised onboarding journeys based on role or team would turn the hub concept into a structured learning experience rather than a flat pile of 'sources'. Richer governance features around retention, moderation, and content lifecycle would make the platform usable for regulated environments.

Each of these represents a natural extension of what the system already supports.

9. Conclusions

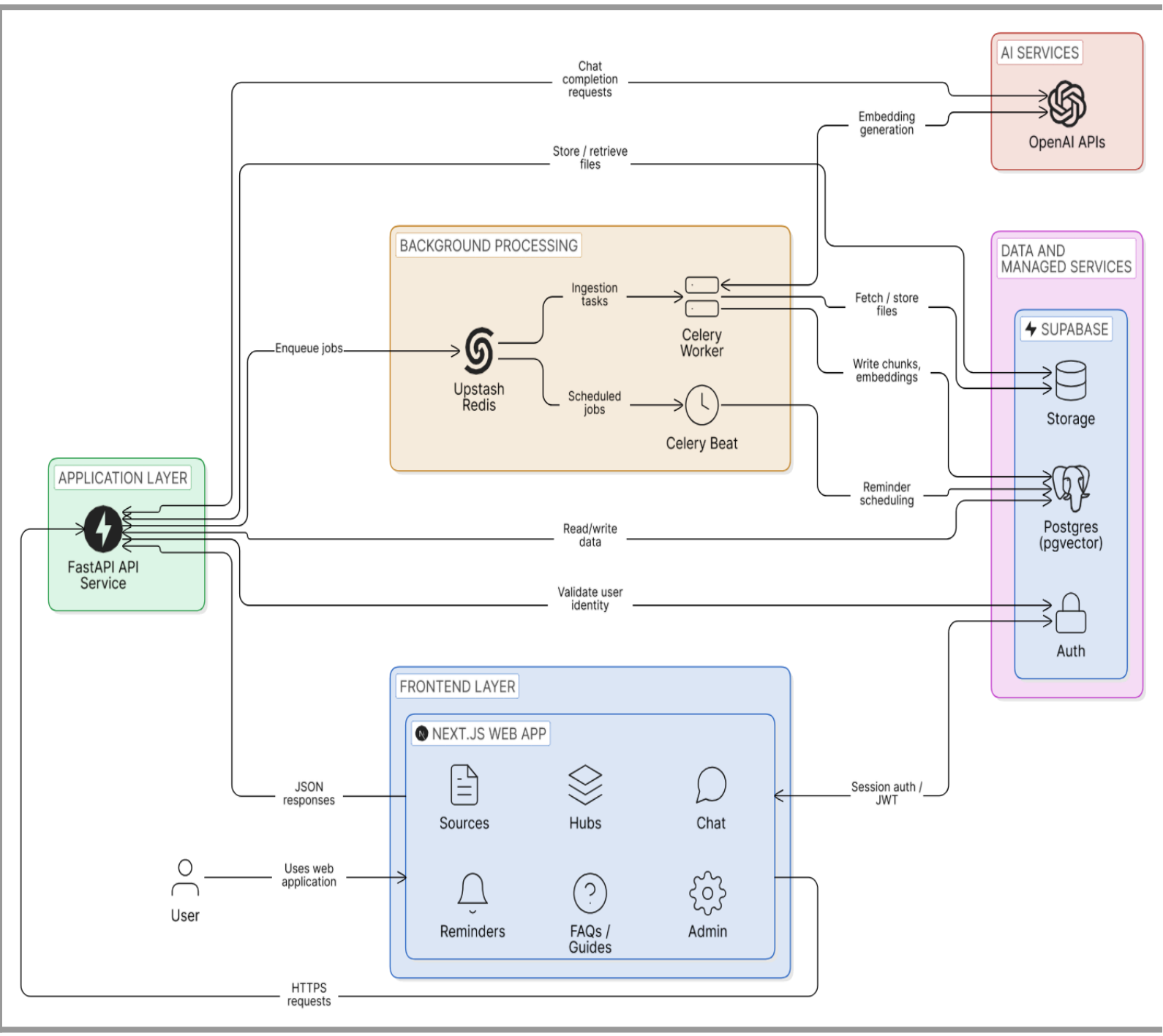
This Technical Guide has shown how Caddie was designed and implemented as a complete documentation support platform. By combining collaborative hubs, asynchronous ingestion, retrieval-backed chat, and generated artefacts such as reminders, FAQs, and guides, the project moves beyond simple document storage and provides a more active way for users to work with knowledge.

A major strength of the project is its architecture. Separating the web application, API, worker, and managed services gave each part a clear responsibility and made the system easier to develop, test, and extend. Design choices such as worker-based ingestion, citation-grounded retrieval, and hub-based collaboration were especially important, as they shaped both the reliability of the system and its future potential. Overall, Caddie meets its technical and functional goals as we defined in the functional specification, and provides a strong foundation for further development.

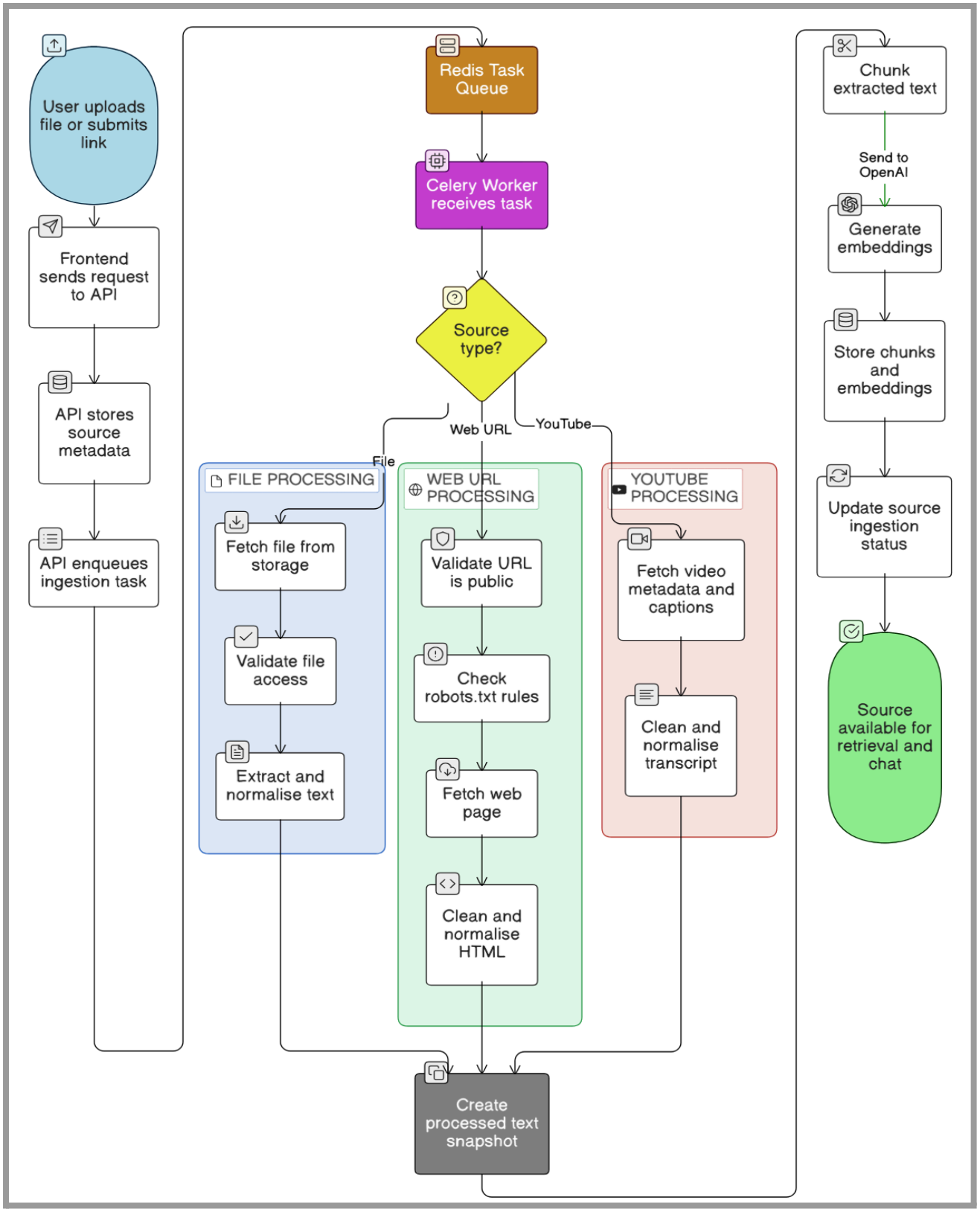
10. Appendices

All architectural diagrams were made using draw.io (https://www.drawio.com/)

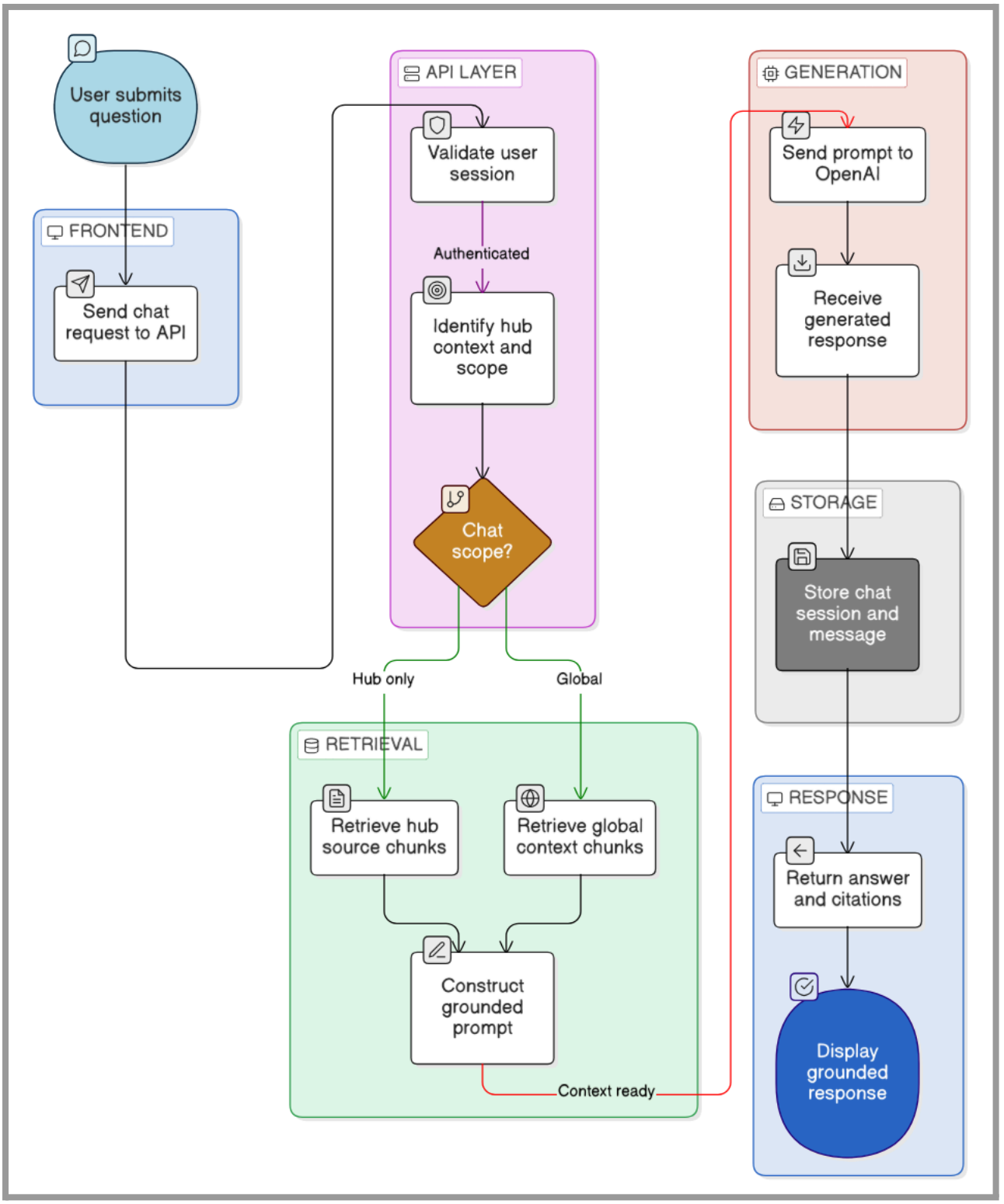
10.1. System Architecture Diagram

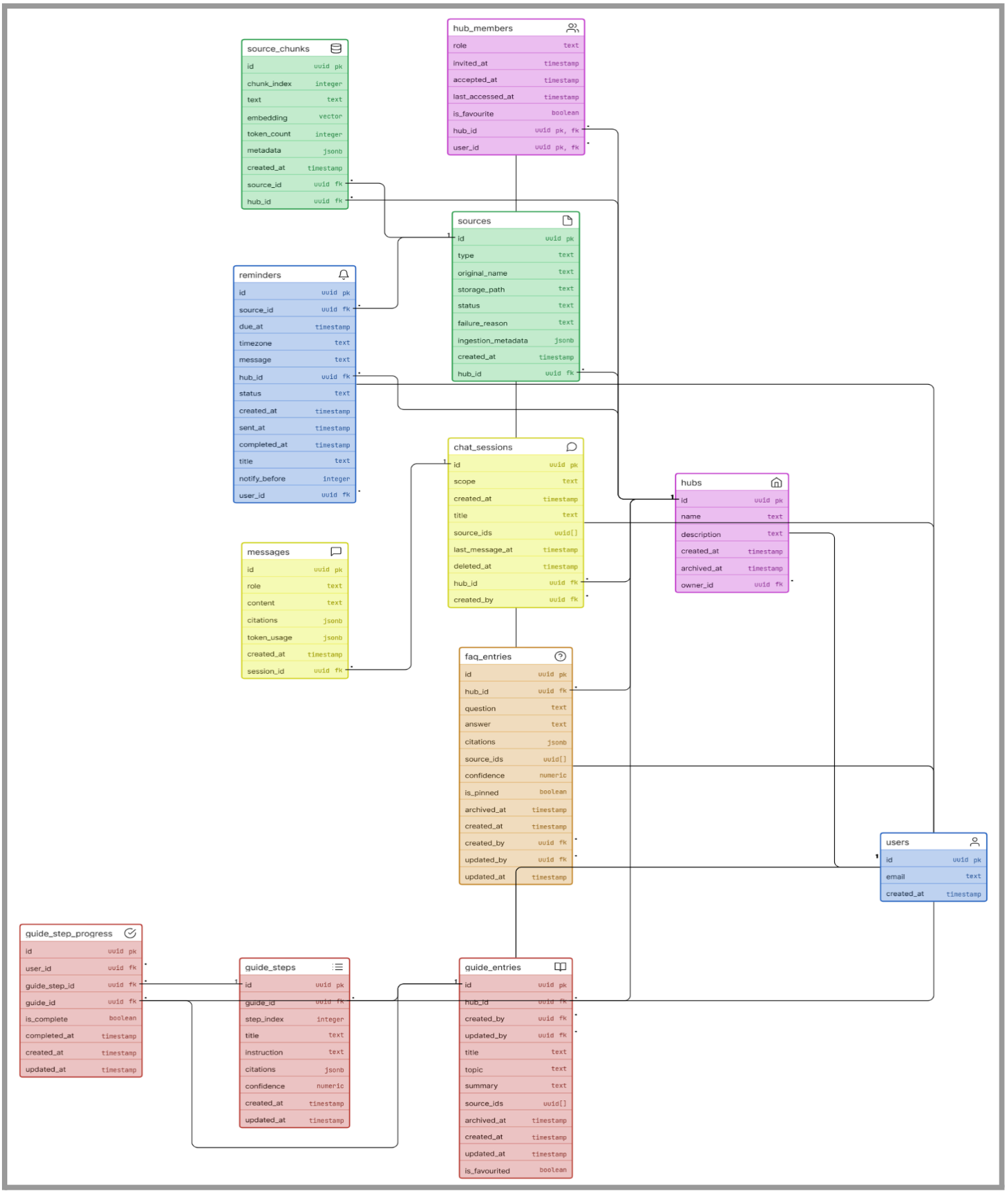


10.2. Asynchronous Worker Processing Pipeline for Source Ingestion



10.3. Chat Request and Retrieval Flow



10.4. Database Schema

Simplified Entity Relationship Diagram highlighting the key tables and how they relate.